

Databázy a vizualizácia

MongoDB

doc. Ing. Ladislav Körösi, PhD.

Ústav robotiky a kybernetiky
FEI STU Bratislava

Relačné databázy:

- pevná schéma (tabuľky, stĺpce),
- normalizácia,
- JOIN operácie.

Problémy:

- zmena štruktúry je náročná,
- veľa JOIN operácií pri čítaní,
- nevhodné pre flexibilné alebo neštruktúrované dáta.

MongoDB:

- dokumentová databáza (JSON/BSON),
- flexibilná štruktúra,
- preferuje vnorené dáta namiesto JOIN.

Základné pojmy

- databáza (database),
- kolekcia (collection) – analógia tabuľky,
- dokument (document) – analógia riadku (záznam).

Dokument:

- dvojice **klúč – hodnota**,
- formát podobný JSON.

Príklad:

```
{  
  meno: "Janko",  
  vek: 20,  
  student: true  
}
```

Dokument môže obsahovať:

- pole (array),
- vnorený objekt.

Príklad:

```
{  
  meno: "Janko",  
  predmety: ["DB", "AI"],  
  adresa: {  
    mesto: "Bratislava",  
    ulica: "Ilkovicova"  
  }  
}
```

Výber databázy:

```
use skola;
```

Databáza sa vytvorí až pri vložení prvých dát do kolekcie studenti (v databáze skola):

```
db.studenti.insertOne({meno: "Janko"});
```

Poznámky:

- MongoDB nemá príkaz CREATE DATABASE,
- databáza existuje až keď obsahuje dáta,
- skola = databáza,
- studenti = kolekcia v databáze.

```
db.studenti.insertOne({  
  meno: "Janko",  
  vek: 20,  
  predmety: ["DB", "AI"]  
});
```

- kolekcia sa vytvorí automaticky,
- MongoDB pridá pole `_id` (primárny kľúč).

Bez formátovania:

```
db.studenti.find();
```

```
{_id: ObjectId("..."), meno: "Janko", vek: 20}
```

Výstup je v jednom riadku (horšie čitateľný). S formátovaním:

```
db.studenti.find().pretty();
```

Výstup je prehľadný:

```
{
  _id: ObjectId("..."),
  meno: "Janko",
  vek: 20
}
```

Jednoduchá podmienka:

```
db.studenti.find({meno: "Janko"});
```

Porovnanie:

```
db.studenti.find({vek: {$gt: 18}});
```

AND podmienka:

```
db.studenti.find({  
  meno: "Janko",  
  vek: {$gt: 18}  
});
```

- podobné ako WHERE v SQL,
- operátory: \$gt, \$lt, \$eq, ...

Príkaz insertMany namiesto insertOne:

```
db.studenti.insertMany([  
  {meno: "Anna", vek: 22},  
  {meno: "Peter", vek: 19}  
]);
```

Dokumenty nemusia mať rovnakú štruktúru.

Aktualizácia jedného záznamu príkazom `updateOne`

```
db.studenti.updateOne(  
  {meno: "Janko"},  
  {$set: {vek: 21}}  
);
```

```
db.studenti.insertOne({  
  meno: "Eva",  
  adresa: {  
    mesto: "Bratislava",  
    ulica: "Ilkovicova"  
  }  
});
```

```
db.studenti.find({"adresa.mesto": "Bratislava"});
```

Bez potreby JOIN operácií.

V MongoDB existujú dva prístupy k návrhu dát:

Embedding (vnorenie):

- dáta sa ukladajú spolu v jednom dokumente,
- rýchle čítanie (bez JOIN),
- vhodné pre úzko súvisiace dáta.

Príklad:

```
{  
  meno: "Janko",  
  predmety: ["DB", "AI"]  
}
```

Referencing (odkazovanie):

- dáta sú v rôznych kolekciách,
- používa sa referencia (napr. ID),
- podobné ako cudzie kľúče v SQL.

Príklad pre kolekciu studenti a kolekciu znamky:

```
{ _id: 1, meno: "Janko" }
```

```
{ student_id: 1, predmet: "DB", znamka: "A" }
```

Pravidlo:

Často čítané dáta spolu → embedding.

Nezávislé alebo veľké dáta → referencing.

Používajú sa v MongoDB indexy?

Indexy zrýchľujú vyhľadávanie v kolekcii.

Vytvorenie indexu:

```
db.studenti.createIndex({meno: 1});
```

- 1 = vzostupné triedenie (ascending),
- -1 = zostupné triedenie (descending),
- podobné ako indexy v SQL.

Bez indexu:

- prehľadáva sa celá kolekcia (full scan).

S indexom:

- rýchle vyhľadávanie podľa daného poľa.

Relačné databázy:

- normalizácia,
- viac tabuliek,
- JOIN operácie.

MongoDB:

- dáta sa ukladajú spolu,
- využíva sa vnorenie (embedding),
- minimalizujú sa JOIN operácie,
- aj MongoDB má indexy.

MongoDB optimalizujeme návrhom dokumentu, nie normalizáciou.